Informalizing Unit with Informath

Gauss's lemma and gcd domains: Unit, Dedukti, English

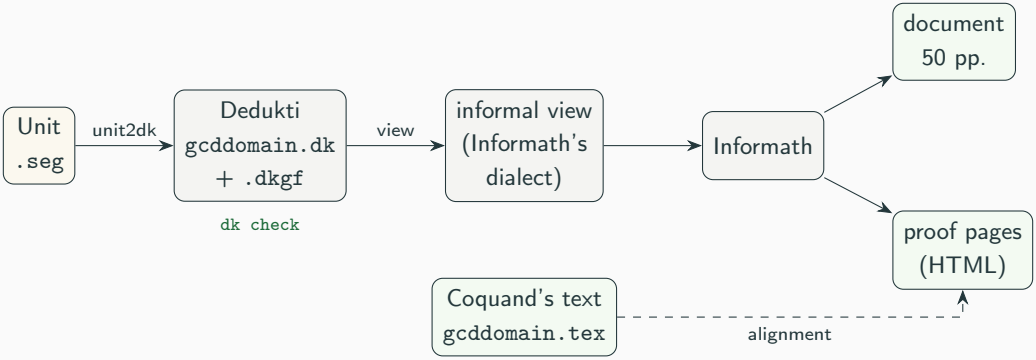
Aarne Ranta, with Claude Code

November 2026

The task

- **Unit** (Thierry Coquand): a dependent type checker with segments (modules), records, universe levels, data types with `split`
- The request: translate Unit to **Dedukti**, preserving type-correctness and the segment structure, so that **Informath** can read Unit developments as it reads Mathlib, Edukera, Godement
- The test case: `unit/sag/gcddomain`, 23 files, 2414 lines
  - Gauss's lemma, and "if A is a gcd domain, then so is $A[X]$"
  - with an informal text by Thierry, `gcddomain.tex`: a *level C* proof in the sense of van Benthem Jutting, written file by file to test the claim that Unit's proof terms have the structure of a detailed informal proof
- So the informalization can be compared with a human text, proof by proof

The pipeline



- **unit2dk** (Haskell): uses Unit's own checker as a library and Informath's Dedukti syntax; reads Unit's values back as typed Dedukti terms
- **Python scripts**: `informal_view.py`, `informal_doc.py`, `proof_sample.py`, `align.py`

From Unit to Dedukti

The encoding (`prelude.dk`)

- **Levels**: `Lvl`, `lz`, `ls`, and `defac lmax` with absorption rules
- **Universes** à la Tarski: codes `U l`, decoding `El l`, cumulativity by `lift`
- **Π, Σ, Id** as codes; `J` computing on `refl`
- **Data types**: constants; `split` as rewrite rules
- **Segments**: lambda-lifted; `#STRUCTURE` lines in the `.dkgf`

```
defac lmax [Lvl].
[l, r] lmax l (lmax (ls l) r) --> lmax (ls l) r.
[l, r] lmax l (ls (lmax l r)) --> ls (lmax l r).

U : Lvl -> Type.
def El : l : Lvl -> U l -> Type.
def pi : l1 : Lvl -> l2 : Lvl -> a : U l1 ->
  (El l1 a -> U l2) -> U (lmax l1 l2).
[l1, l2, a, b] El _ (pi l1 l2 a b)
  --> x : El l1 a -> El l2 (b x).
```

Records: the typed encoding

```
record CRing {
  R : U a ;
  z : R ; o : R ;
  add : R -> R -> R ;
  mul : R -> R -> R ;
  ...
  mul0 (x : R) : Id R (mul x o) x ;
  ...
} ;
module RL (Rg : CRing) { open Rg ; ... }
```

```
CRing    : a : Lvl -> U (ls a).
CRing.mk : a : Lvl -> R : U a -> z : El a R
  -> ... -> El (ls a) (CRing a).
def CRing.mul : a : Lvl -> t : El (ls a) (CRing a)
  -> El a (CRing.R a t) -> ... .
[a, R, z, ...] CRing.mul a (CRing.mk a R z ...)
  --> mul.
```

- a type, a constructor, a *selector* per field with its rule; derived members take the element; nominal, no η, as in Unit
- `.dkgf`: `#RECORD CRing CRing.mk CRing.R ...`, for Informath to read the constructor as a structure definition
- **Fixed fields**: `isEquiv A B f = IsEquiv[A, B, f]` is a record of its own, `IsEquiv[A,B,f]`, with a coercion `.up` to `IsEquiv`; the variants used are found by translating again until none is missing

Results on `sag/gcddomain`

	first version	now
declarations translated in full	111	612
as axioms	55	0
left out	444	0
accepted by `dk check`	—	689 declarations, 220 rule sets
demoted	—	1

- The one demoted: `FunExtUA.FE.totC`, which needs η for Σ on neutral terms: Dedukti's matching does not see `pair (x => fst p x) (x => snd p x)` as `p`
- Translation time: 0.1 s (twice, for the variants of records)

Found and fixed on the way

- **Unit changed under us**: indexed inductive families arrived (5 Nov); the translator adapted, and a misreading of mine (taking every parametrized data type for a family) cost 500 declarations until fixed
- **Levels**: `lmax` is AC, but rules like `lmax l (ls l) -> ls l` match only a whole join; rules with a variable for the rest, and absorption under a successor, were needed
- **Parameter types at instances**: reading a type back and evaluating it again turned `PolyR.P` into a rigid constant; the type is now evaluated from its syntax
- **Dedukti's grammar**: `(f : A -> B)` is the type `(f : A) -> B`, in the official grammar too; a field of function type is written `(f : (A -> B))`
- **Informath**: now parses `defac` / `defacu`

From Dedukti to English

The informal view

unit2dk's Dedukti is checkable, not readable. `informal_view.py` rewrites it into Informath's dialect (`Set`, `Elem`, `Proof`, `forall`, `exists`, ...):

- **dropped**: levels, lifts, and the foundations' parameters (`univ`, `images`, `surjections`)
- `El` **of a code**: a function type, an equation `Proof (Id A x y)`, a proposition `Proof P`, or `Elem A`
- **propositions** are recognized: identity types, `Empty`, `Not`, sums of propositions (decisions), Π and Σ of propositions (*for all*, *there exists*), and constants defined as such (`Div`, `GcdOf`)
- **records** are structures; their selectors become notation; binary operations are `Oper` fields named by their symbols
- **names** shortened: `div_trans`, `Gcd_gcd`

`gcddomain-view.dkgf`: about 50 symbol table entries in the notation of Thierry's text: $a \mid b$, $\mathrm{c}(f)$, p_k, $a + Xp$, $A[X]$; one new word in `Mathlib.gf`: *gcd domain*

Readings

Structure. A commutative ring is an algebraic structure, where R is a set, z is an element of R, ..., $+$ is a binary operation on R, ..., for all elements x, y and w of R, $x + (y + w) = (x + y) + w$, ...

Definition. Let A be a commutative ring. Let a and b be elements of R. Then a divides b, if there exists an element q of R such that $b = aq$.

Div_trans. Let a, b and c be elements of R such that a divides b and b divides c. Then a divides c.

Gauss. Let p and q be elements of $\mathcal{P}$ such that the function that maps k to p_k is primitive. Let d be an element of R such that d is a common divisor of the coefficients of pq. Let i be a natural number. Then d divides q_i.

The main theorem. Let A be a commutative ring. Let G be a gcd domain on A. Then `Over_domain` applied to A and G is a gcd domain on $A[X]$.

The document

`informal_doc.py`: one section per segment, its parameters said once, as an informal text does:

GcdDomain.Domain
In this section: Let A be a commutative ring. Assume that R is a set. Assume that for all elements x of R, $x = 0$ or x is not equal to 0. Assume that for all x and y ...

Sub_zero. Let a and b be elements of R such that $a + -b = 0$. Then $a = b$.

- 635 of 636 declarations read, 57 sections, 50 pages, LaTeX without errors
- the section's parameters come from the `#STRUCTURE` lines of unit2dk's `.dkgf`; the context sentence is Informath's reading of them as hypotheses

Proofs

Proof texts

Informath's `-proof-text` reads a proof term as numbered lines, each justified by the rule it applies. For Unit's proofs, the view writes the rules (types in `unit-nd.dk`):

- a pair proving a Σ of a proposition: `existsI`; its projections `fst`, `snd`
- `J` at a motive not using the path (the steps of `calc`): `trans`, or `transport`
- a hypothesis applied to arguments: `forallE`, `ifE`, unfolding defined propositions (`Rec N`, `Common d f`)
- a λ proving *for all* or *if*: `forallI`, `ifI`

div-trans, with `-force-symbolic`:

1.	Assume that $a \mid b$.	
2.	Assume that $b \mid c$.	
3.	$c = b\,\pi_1(k)$.	`snd 2`
...		
9.	$c = a(\pi_1(h)\,\pi_1(k))$.	`trans 3, 8`
10.	There exists an element q of R, such that $c = aq$.	`existsI 9`

The proof pages

`proof_sample.py`: for each proof, a table of its steps, numbered as the proof text numbers its lines (the hypotheses first, then each step after its premisses; checked against the lines' rules):

#	Informath	by	Unit
3	$a \mid ca$.	`div_factor_left`	`div-factor-left a c`
4	$bc = cb$.	`mulC`	`mulC b c`
5	$a \mid cb$.	`div_eq_right 4, 2`	`div-eq-right a (mul b c) (mul c b) 4 2`
6	$a \mid c \gcd(a, b)$.	`lemma_M 3, 5`	`lemma-M c a b a 3 5`
...			
10	By div_trans, $a \mid c$.	`div_trans 6, 9`	`div-trans a (mul c (gcd a b)) c 6 9`

- a step is its rule applied to its other arguments, the numbers of its premisses (links) in place of their proofs, indented by depth
- the Unit column omits what Unit's source omits (implicit arguments, module parameters): it reproduces the source of `euclid`
- a Dedukti column, hidden behind a toggle; the Unit source and the whole proof term above the table

Aligning Thierry's text with the steps

`align.py`, automatic:

1. **find the text** by `\nm{name}`: a lemma titled with it, an item of a lemma's list and the paragraph "(k)" of its proof, or an item of a module's proof (`Tail.eq`)
2. **cut it** at its formulas; chains $c = bk = (ah)k$ at their relations
3. **parse the formulas** (|, =, products, sums, gcd, $\mathrm{c}(p)$, q_{i+1}, $a + Xp$) and normalize the steps' conclusions the same way
4. **link** a fragment to the steps whose conclusions it states: equations either way round; $\pi_1(h)$ as h; a letter that is not a variable of the theorem (a quotient q) matching any term; polynomial identities by their coefficients; local definitions ($t = \gcd(d, a_0)$) by their values; claims in words ("is a common divisor of", "is primitive"); names of lemmas used

div-trans, (4)

If $b = ah$ [4] and $c = bk$ [3] then $c = bk$ [3] $= (ah)k$ [5] $= a(hk)$ [6–9], so the quotient is hk [10].

Alignment results

sample	proofs	texts aligned with the steps	fragments linked
divisibility: div-refl, div-of-zero, div-trans, euclid	4	4	as by hand, but 1
Gauss's lemma: Tail, PrimStep.One, PrimStep	11	11	23 of 28

- the unlinked fragments: words without formulas, "by the claim for a, p, g", "apply $\mathrm{Rec}(N)$"
- the hand-made alignment of the first sample is kept (`proof-sample.json`) for comparison
- the text's proof of Gauss's lemma is a list of items, one per member of a Unit module: *the proof terms do have the structure of the informal proof*, and the members are its steps

Changes in Informath

- `defac` and `defacu` in the Dedukti grammar, read as declarations of binary operations `b2f9f3c`
- `gcd_domain_Noun` in `Mathlib.gf`, five languages `0a59340`
- `-to-latex-doc` escapes underscores in labels such as `div_trans` `41b41d7`
- `#EXCLUDE` also leaves out the rewrite rules of the excluded constants `e043d27`

Issues found, worked around in the view:

- symbol table entries for names in `{|...|}` are never used (`lookupConstant`)
- a LaTeX macro of a constant used with fewer arguments than its notation breaks the output (the view η-expands)
- Swedish *var* for *where* in structure definitions; `AdjKind` and `DepNoun` only in English and French

What remains

- **Translation**: indexed inductive families (none in gcddomain); Σ-η on neutral terms; Unit's `core` (100 MB of output when local definitions are unfolded)
- **Readings**: witnesses as $\pi_1(h)$ where a text says "let q be such that $b = aq$" (an elimination of *exists*); definitions by `split` (`One.coeffs`) have rules, not proof terms
- **Alignment**: the formula parser and the verbal claims are those of this text; a larger sample, and other texts, will show how far it carries
- **The comparison itself**: Thierry's claim that the proof terms have the structure of the informal proof can now be inspected step by step, on pages that show both

Repository: `github.com/aarneranta/unit-informath`